

Lecture 18

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

September 8, 2026

Example

- Consider a grammar for signed binary numbers
  $Number \rightarrow sign \quad list$
  $sign \rightarrow +|-$
  $list \rightarrow list \quad bit|bit$
  $bit \rightarrow 0|1$

Example

- Consider a grammar for signed binary numbers
  $Number \rightarrow sign \quad list$
  $sign \rightarrow +|-$
  $list \rightarrow list \quad bit|bit$
  $bit \rightarrow 0|1$
- Build attribute grammar that annotates Number with the value it represents

Example

- Consider a grammar for signed binary numbers
  $Number \rightarrow sign \quad list$
  $sign \rightarrow +|-$
  $list \rightarrow list \quad bit|bit$
  $bit \rightarrow 0|1$
- Build attribute grammar that annotates Number with the value it represents
- Associate attributes with grammar symbols

Example

- Consider a grammar for signed binary numbers
  $Number \rightarrow sign \quad list$
  $sign \rightarrow +|-$
  $list \rightarrow list \quad bit|bit$
  $bit \rightarrow 0|1$
- Build attribute grammar that annotates Number with the value it represents
- Associate attributes with grammar symbols

Symbol	Attribute
number	value
sign	negative
list	position, value
bit	position, value

Attribute Rules

Production	Attribute Rule

Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	*list.position* $\leftarrow$ 0
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	*list.position* $\leftarrow$ 0
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

sign $\rightarrow$ +	sign.negative $\leftarrow$ false

Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	*list.position* $\leftarrow$ 0
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

sign $\rightarrow$ +	sign.negative $\leftarrow$ false
sign $\rightarrow$ -	sign.negative $\leftarrow$ true

Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	*list.position* $\leftarrow$ 0
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

sign $\rightarrow$ +	sign.negative $\leftarrow$ false
sign $\rightarrow$ -	sign.negative $\leftarrow$ true
list $\rightarrow$ bit	bit.position $\leftarrow$ list.position list.value $\leftarrow$ bit.value

Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	$list.position \leftarrow 0$
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

sign $\rightarrow$ +	sign.negative $\leftarrow$ false
sign $\rightarrow$ -	sign.negative $\leftarrow$ true
list $\rightarrow$ bit	bit.position $\leftarrow$ list.position list.value $\leftarrow$ bit.value
$list_0 \rightarrow list_1\, bit$	$list_1.position \leftarrow list_0.position + 1$ $bit.position \leftarrow list_0.position$ $list_0.value \leftarrow list_1.value + bit.value$

Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	$list.position \leftarrow 0$
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

sign $\rightarrow$ +	sign.negative $\leftarrow$ false
sign $\rightarrow$ -	sign.negative $\leftarrow$ true
list $\rightarrow$ bit	bit.position $\leftarrow$ list.position list.value $\leftarrow$ bit.value
$list_0 \rightarrow list_1\, bit$	$list_1.position \leftarrow list_0.position + 1$ $bit.position \leftarrow list_0.position$ $list_0.value \leftarrow list_1.value + bit.value$
bit $\rightarrow$ 0	bit.value $\leftarrow$ 0

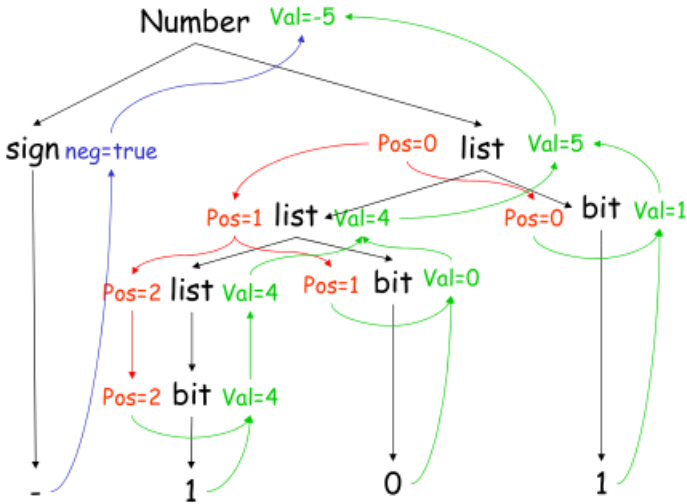
Attribute Rules

Production	Attribute Rule

number $\rightarrow$ sign list	$list.position \leftarrow 0$
	`if sign.negative`
	`then number.value` $\leftarrow$ `- list.value`
	`else number.value` $\leftarrow$ `list.value`

sign $\rightarrow$ +	sign.negative $\leftarrow$ false
sign $\rightarrow$ -	sign.negative $\leftarrow$ true
list $\rightarrow$ bit	bit.position $\leftarrow$ list.position list.value $\leftarrow$ bit.value
$list_0 \rightarrow list_1\, bit$	$list_1.position \leftarrow list_0.position + 1$ $bit.position \leftarrow list_0.position$ $list_0.value \leftarrow list_1.value + bit.value$
bit $\rightarrow$ 0	bit.value $\leftarrow$ 0
bit $\rightarrow$ 1	bit.value $\leftarrow 2^{bit.position}$

Parse tree and the dependence graph



Synthesized Attributes

- A syntax directed definition that uses only synthesized attributes is said to be an S-attributed definition

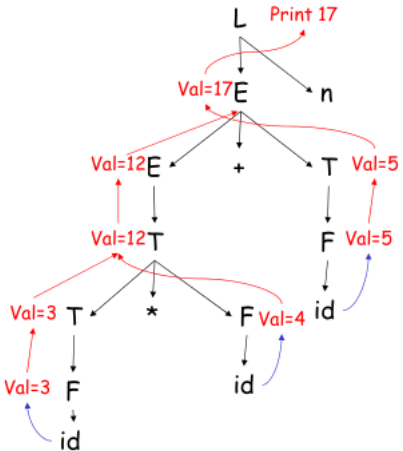
Synthesized Attributes

- A syntax directed definition that uses only synthesized attributes is said to be an S-attributed definition
- A parse tree for an S-attributed definition can be annotated by evaluating semantic rules for attributes

Synthesized Attributes

- A syntax directed definition that uses only synthesized attributes is said to be an S-attributed definition
- A parse tree for an S-attributed definition can be annotated by evaluating semantic rules for attributes
- $L \rightarrow En \qquad Print(E.val)$
  $E \rightarrow E + T \qquad E.val = E.val + T.val$
  $E \rightarrow T \qquad E.val = T.val$
  $T \rightarrow T * F \qquad T.val = T.val * F.val$
  $T \rightarrow F \qquad T.val = F.val$
  $F \rightarrow (E) \qquad F.val = E.val$
  $F \rightarrow digit \qquad F.val = digit.lexval$

Parse tree for 3 * 4 + 5 n



Inherited Attributes

- An inherited attribute is one whose value is defined in terms of attributes at the parent and/or siblings

Inherited Attributes

- An inherited attribute is one whose value is defined in terms of attributes at the parent and/or siblings
- Used for finding out the context in which it appears

Inherited Attributes

- An inherited attribute is one whose value is defined in terms of attributes at the parent and/or siblings
- Used for finding out the context in which it appears
- Possible to use only S-attributes but more natural to use inherited attributes

Inherited Attributes

- An inherited attribute is one whose value is defined in terms of attributes at the parent and/or siblings
- Used for finding out the context in which it appears
- Possible to use only S-attributes but more natural to use inherited attributes

$D \rightarrow TL \qquad L.in = T.type$

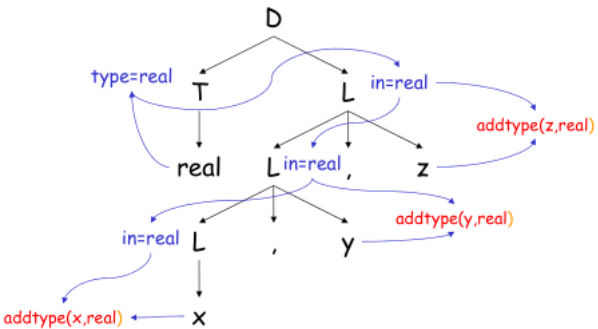
$T \rightarrow real \qquad T.type = real$

$T \rightarrow int \qquad T.type = int$

$L \rightarrow L_1, id \qquad L_1.in = L.in; addtype(id.entry, L.in)$

$L \rightarrow id \qquad addtype(id.entry, L.in)$

Parse tree for real x, y, z



Dependency Graph

- If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c

Dependency Graph

- If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c
- The dependencies among the nodes can be depicted by a directed graph called dependency graph

Dependency Graph

- If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c
- The dependencies among the nodes can be depicted by a directed graph called dependency graph
- Algorithm:

Dependency Graph

- If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c
- The dependencies among the nodes can be depicted by a directed graph called dependency graph
- Algorithm:

```
for each node n in the parse tree do
    for each attribute a of the grammar symbol do
        construct a node in the dependency graph for a
```

Dependency Graph

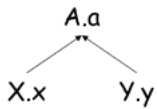
- If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c
- The dependencies among the nodes can be depicted by a directed graph called dependency graph
- Algorithm:

```
for each node n in the parse tree do
     for each attribute a of the grammar symbol do
         construct a node in the dependency graph for a
for each node n in the parse tree do
     for each semantic rule b = f(c_1, c_2, ..., c_k) do
         for i = 1 to k do
             construct an edge from c_i to b
```

Example

- Suppose $A.a = f(X.x, Y.y)$ is a semantic rule for $A \rightarrow XY$





Example

- Suppose $A.a = f(X.x, Y.y)$ is a semantic rule for $A \rightarrow XY$



- If production $A \rightarrow XY$ has the semantic rule $X.x = g(A.a, Y.y)$



Example

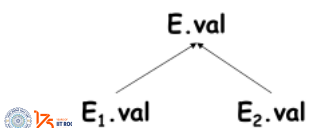
- Suppose $A.a = f(X.x, Y.y)$ is a semantic rule for $A \rightarrow XY$



- If production $A \rightarrow XY$ has the semantic rule $X.x = g(A.a, Y.y)$



- Whenever following production is used in a parse tree
  $E \rightarrow E_1 + E_2 \quad E.val = E_1.val + E_2.val$



Abstract Syntax Tree

- Condensed form of parse tree

Abstract Syntax Tree

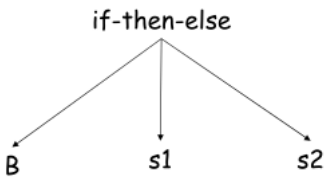
- Condensed form of parse tree
- Useful for representing language constructs.

Abstract Syntax Tree

- Condensed form of parse tree
- Useful for representing language constructs.
- The production `S` $\rightarrow$ `if B then s1 else s2` may appear as

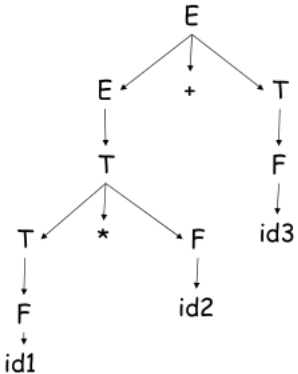
Abstract Syntax Tree

- Condensed form of parse tree
- Useful for representing language constructs.
- The production S $\rightarrow$ `if B then s1 else s2` may appear as



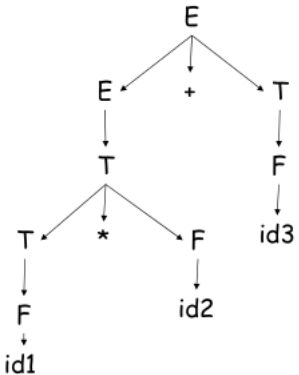
Abstract Syntax Tree

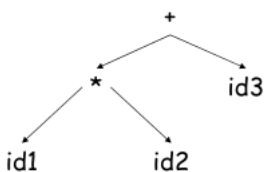
- Chain of single productions may be collapsed, and operators move to the parent nodes



Abstract Syntax Tree

- Chain of single productions may be collapsed, and operators move to the parent nodes





Constructing Abstract Syntax tree for expression

- Each node can be represented as a record

Constructing Abstract Syntax tree for expression

- Each node can be represented as a record
- **Operators:** one field for operator, remaining fields ptrs to operands
  mknode(op,left,right)

Constructing Abstract Syntax tree for expression

- Each node can be represented as a record
- **Operators:** one field for operator, remaining fields ptrs to operands
  mknode(op,left,right)
- **identifier:** one field with label id and another ptr to symbol table
  mkleaf(id,entry)

Constructing Abstract Syntax tree for expression

- Each node can be represented as a record
- **Operators:** one field for operator, remaining fields ptrs to operands
  mknode(op,left,right)
- **identifier:** one field with label id and another ptr to symbol table
  mkleaf(id,entry)
- **number:** one field with label num and another to keep the value of the number
  mkleaf(num,val)

Example

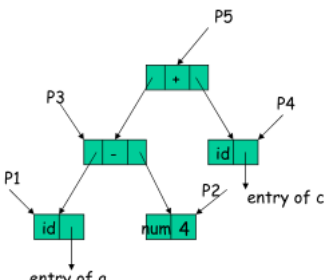
- The following sequence of function calls creates a parse tree for $a - 4 + c$

Example

- The following sequence of function calls creates a parse tree for $a - 4 + c$
  $P_1 = mkleaf(id, entry.a)$
  $P_2 = mkleaf(num, 4)$
  $P_3 = mknode(-, P_1, P_2)$
  $P_4 = mkleaf(id, entry.c)$
  $P_5 = mknode(+, P_3, P_4)$

Example

- The following sequence of function calls creates a parse tree for $a - 4 + c$
  $P_1 = mkleaf(id, entry.a)$
  $P_2 = mkleaf(num, 4)$
  $P_3 = mknode(-, P_1, P_2)$
  $P_4 = mkleaf(id, entry.c)$
  $P_5 = mknode(+, P_3, P_4)$



A syntax directed definition for constructing syntax tree

$E \rightarrow E_1 + T$	$E.ptr = mknode(+, E_1.ptr, T.ptr)$
$E \rightarrow T$	$E.ptr = T.ptr$
$T \rightarrow T_1 * F$	$T.ptr := mknode(*, T_1.ptr, F.ptr)$
$T \rightarrow F$	$T.ptr := F.ptr$
$F \rightarrow (E)$	$F.ptr := E.ptr$
$F \rightarrow id$	$F.ptr := mkleaf(id, entry.id)$
$F \rightarrow num$	$F.ptr := mkleaf(num, val)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

$P_5 = makenode(-, P3, P4)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

$P_5 = makenode(-, P3, P4)$

$P_6 = makenode(*, P_2, P_5)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

$P_5 = makenode(-, P3, P4)$

$P_6 = makenode(*, P_2, P_5)$

$P_7 = makenode(+, P_1, P_6)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

$P_5 = makenode(-, P3, P4)$

$P_6 = makenode(*, P_2, P_5)$

$P_7 = makenode(+, P_1, P_6)$

$P_8 = makeleaf(id, b)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

$P_5 = makenode(-, P3, P4)$

$P_6 = makenode(*, P_2, P_5)$

$P_7 = makenode(+, P_1, P_6)$

$P_8 = makeleaf(id, b)$

$P_9 = makeleaf(id, c)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$

$P_2 = makeleaf(id, a)$

$P_3 = makeleaf(id, b)$

$P_4 = makeleaf(id, c)$

$P_5 = makenode(-, P3, P4)$

$P_6 = makenode(*, P_2, P_5)$

$P_7 = makenode(+, P_1, P_6)$

$P_8 = makeleaf(id, b)$

$P_9 = makeleaf(id, c)$

$P_{10} = makenode(-, P_8, P_9)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$
$P_2 = makeleaf(id, a)$
$P_3 = makeleaf(id, b)$
$P_4 = makeleaf(id, c)$
$P_5 = makenode(-, P3, P4)$
$P_6 = makenode(*, P_2, P_5)$
$P_7 = makenode(+, P_1, P_6)$
$P_8 = makeleaf(id, b)$
$P_9 = makeleaf(id, c)$
$P_{10} = makenode(-, P_8, P_9)$
$P_{11} = makeleaf(id, d)$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$
$P_2 = makeleaf(id, a)$
$P_3 = makeleaf(id, b)$
$P_4 = makeleaf(id, c)$
$P_5 = makenode(-, P3, P4)$
$P_6 = makenode(*, P_2, P_5)$
$P_7 = makenode(+, P_1, P_6)$
$P_8 = makeleaf(id, b)$
$P_9 = makeleaf(id, c)$
$P_{10} = makenode(-, P_8, P_9)$
$P_{11} = makeleaf(id, d)$
$P_{12} = makenode(*, P_{10}, P_{11})$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$
$P_2 = makeleaf(id, a)$
$P_3 = makeleaf(id, b)$
$P_4 = makeleaf(id, c)$
$P_5 = makenode(-, P3, P4)$
$P_6 = makenode(*, P_2, P_5)$
$P_7 = makenode(+, P_1, P_6)$
$P_8 = makeleaf(id, b)$
$P_9 = makeleaf(id, c)$
$P_{10} = makenode(-, P_8, P_9)$
$P_{11} = makeleaf(id, d)$
$P_{12} = makenode(*, P_{10}, P_{11})$
$P_{13} = makenode(+, P_7, P_{12})$

DAG for Expressions

Expression a + a * (b – c) + (b - c) * d

make a leaf or node if not present, otherwise return pointer to the existing node

$P_1 = makeleaf(id, a)$
$P_2 = makeleaf(id, a)$
$P_3 = makeleaf(id, b)$
$P_4 = makeleaf(id, c)$
$P_5 = makenode(-, P3, P4)$
$P_6 = makenode(*, P_2, P_5)$
$P_7 = makenode(+, P_1, P_6)$
$P_8 = makeleaf(id, b)$
$P_9 = makeleaf(id, c)$
$P_{10} = makenode(-, P_8, P_9)$
$P_{11} = makeleaf(id, d)$
$P_{12} = makenode(*, P_{10}, P_{11})$
$P_{13} = makenode(+, P_7, P_{12})$

